Modifiers, Spacing and Responsive Behavior Report

Jetpack Compose Layout Behavior Analysis

Submitted By

Priyanka K

Internship Course

Android App Development using Gen AI

Task 3 Submission

February 19, 2026

1. Introduction

Jetpack Compose uses Modifiers to define layout structure, spacing, size, alignment, and visual styling. This report documents four experiments conducted to understand how modifiers influence layout behavior and responsiveness across screen sizes.

2. Experiment 1 – Padding Modifier

Expected Behavior

Applying `padding(24.dp)` should create internal spacing around the composable content, preventing it from touching screen boundaries.

Observed Behavior

After applying padding, visible space appeared around the text. The layout looked more readable and less crowded.

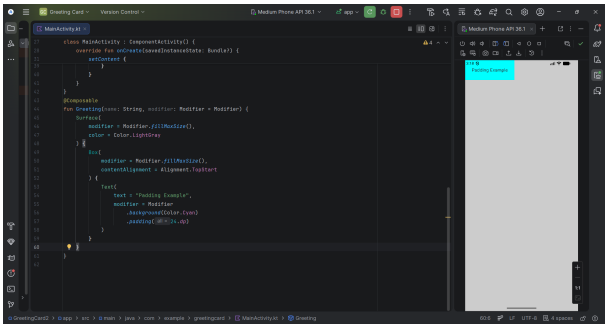

Figure 1: Padding experiment showing visible spacing around text content.

Conclusion

Padding improves layout readability and ensures consistent spacing between UI elements.

3. Experiment 2 – fillMaxWidth() and weight()

Expected Behavior

Using `fillMaxWidth()` should expand the Row to full screen width, while `weight()` should distribute available space proportionally among child elements.

Observed Behavior

The Row occupied the entire width of the screen. The first box (weight = 1f) occupied one portion of space, while the second box (weight = 2f) occupied twice the width, confirming proportional distribution.

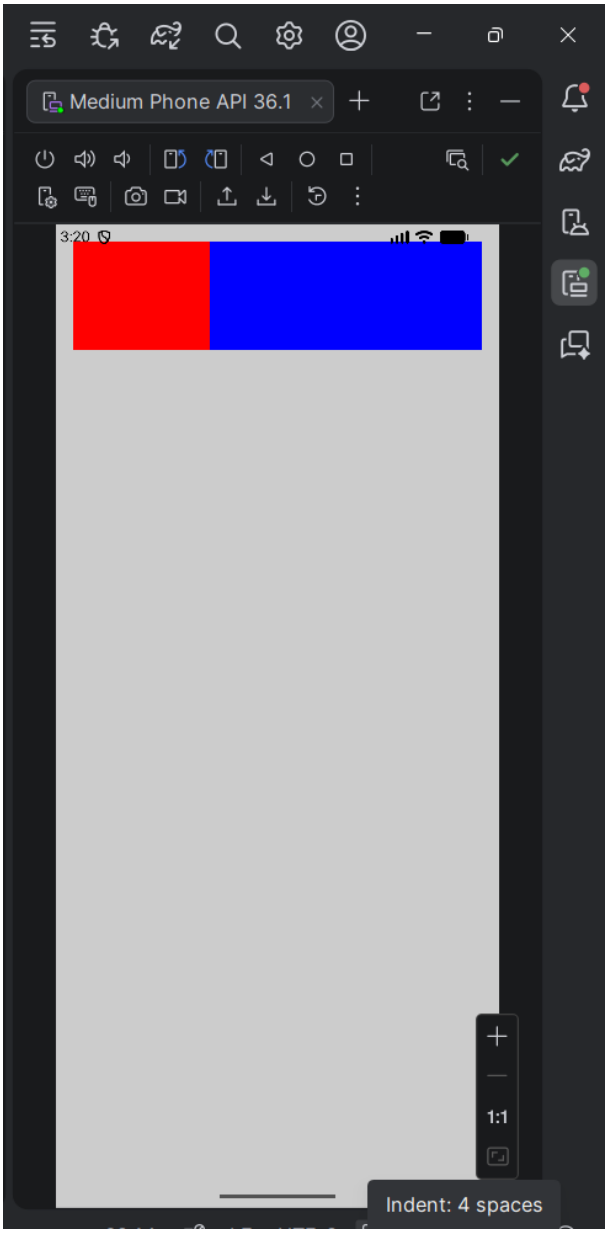

Figure 2: Weight modifier demonstrating proportional width distribution.

Conclusion

The weight modifier enables responsive layouts without relying on fixed dimensions.

4. Experiment 3 – Arrangement.SpaceBetween

Expected Behavior

`Arrangement.SpaceBetween` should distribute elements evenly across available horizontal space.

Observed Behavior

The first element aligned to the extreme left, the last aligned to the extreme right, and remaining space was evenly distributed between elements.

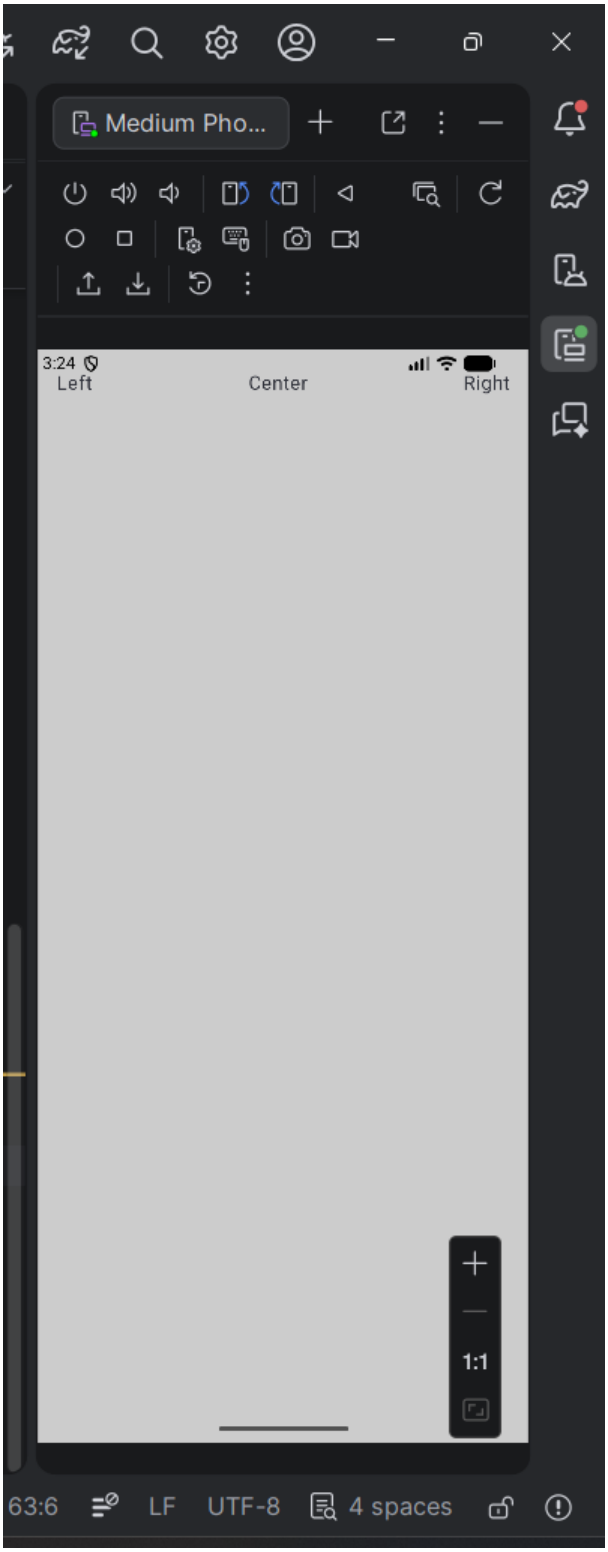

Figure 3: Arrangement.SpaceBetween distributing elements across the Row.

Conclusion

Arrangement.SpaceBetween ensures balanced horizontal spacing and improves layout alignment consistency.

5. Experiment 4 – Modifier Order

Expected Behavior

Changing the order of modifiers should produce visually different results, even when using the same modifiers.

Observed Behavior

When `padding()` was applied before `background()`, the background color covered only the content area. When `background()` was applied before `padding()`, the background expanded to include the padded region.

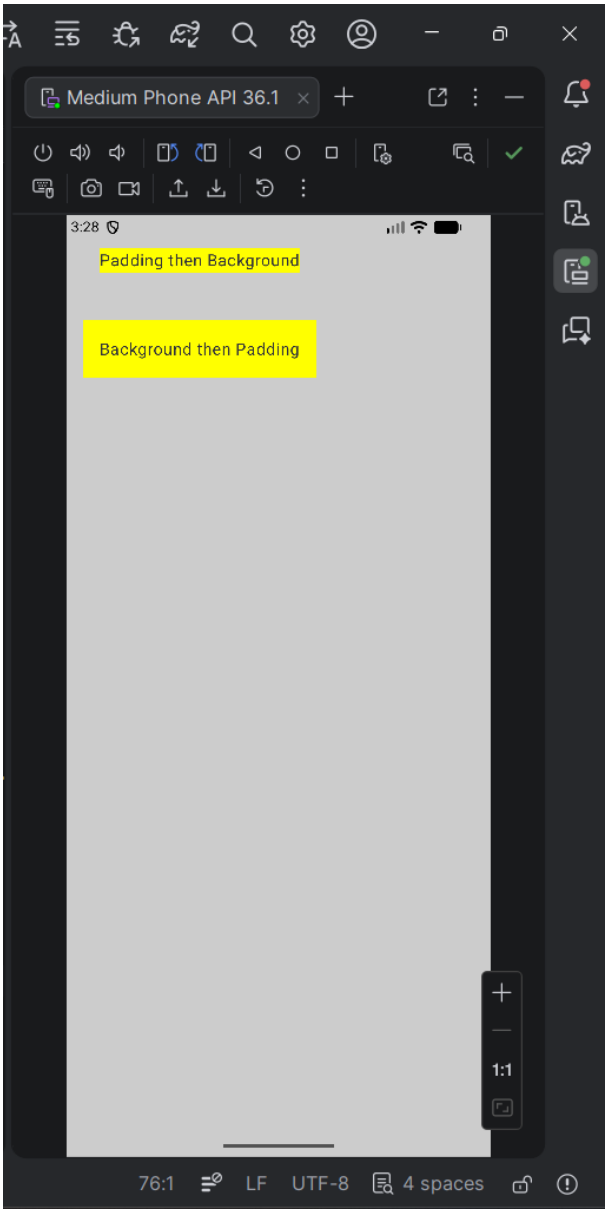

Figure 4: Comparison demonstrating how modifier order affects layout rendering.

Conclusion

Modifiers in Jetpack Compose are executed sequentially from left to right. Changing their order directly affects layout behavior and visual output.

6. Responsive Design Suggestions

- Use `weight()` instead of fixed widths to create adaptable layouts.
- Avoid hardcoded dimensions where possible.
- Test layouts on multiple screen sizes before final deployment.

7. Accessibility Consideration

Proper spacing improves usability and accessibility. Maintaining sufficient padding and a minimum touch target size of 48dp ensures better interaction and readability.

8. Conclusion

The experiments confirm that modifiers significantly influence layout structure, spacing, and responsiveness. A clear understanding of modifier behavior is essential for designing flexible, accessible, and visually consistent Jetpack Compose interfaces.